

GitHub Actions & CI/CD

Workflows, Jobs, Steps, Runners Autohosteados y Despliegue

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI

- sep=10pt **01.** Fundamentos y Arquitectura de GitHub Actions
- sep=20pt **02.** Jerarquía de Componentes: Triggers, Jobs, Stages y Steps
- sep=30pt **03.** Comandos Shell (`run:`) vs. Acciones Reutilizables (`uses:`)
- sep=40pt **04.** Comparativa Exhaustiva: **GitHub-Hosted vs. Self-Hosted Runners**
- sep=50pt **05.** Guía Paso a Paso: Construcción del Pipeline CI/CD del Taller

01

¿Qué es GitHub Actions?

Motor de Automatización y CI/CD Integrado

¿Qué es GitHub Actions?

- **Plataforma CI/CD Nativa:** Integrada directamente en el repositorio sin servidores externos.
- **Basada en Eventos:** Reacciona a sucesos del ciclo Git (`push`, `pull_request`, etc.).
- **Runners Aislados:** VMs efímeras provistas por GitHub (`ubuntu-latest`, `windows-latest`, `macos-latest`).
- **Marketplace:** Miles de acciones oficiales y comunitarias listas para reutilizar.

Principio Clave

Shift-Left Testing:

Detectar y corregir defectos en etapas tempranas mediante builds y tests automáticos en cada *commit*.

Entidades Principales

- **Workflow:** Archivo YAML en `.github/workflows/`.
- **Events (on):** Disparadores del flujo.
- **Jobs:** Tareas en un runner independiente.
- **Steps:** Pasos atómicos secuenciales.

Ciclo de Vida

1. Evento ocurre en Git (*Push* o *PR*).
2. GitHub asigna una Máquina Virtual (*Runner*).
3. Se ejecutan los Jobs (paralelo o secuencial).
4. Se reporta el estado (*Pass / Fail*).

02

Jerarquía de Componentes

Triggers, Jobs, Stages y Steps

1. Triggers y Eventos (on:)

El bloque on: define *cuándo* debe dispararse la ejecución del workflow.

```
name: Continuous Integration & Delivery

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch: # Permite disparo manual desde la UI web
```

- **Filtros de rama:** Limita la ejecución a ramas estratégicas (main, develop).
- **Filtros por ruta (paths):** Dispara el job solo si cambian carpetas específicas.

2. Jobs y Runners (jobs: <job_id>)

Cada **Job** corre en una máquina virtual limpia. Por defecto, se ejecutan en **paralelo**.

```
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Maven Tests
        run: cd backend && mvn clean test
```

- runs-on: Especifica el SO del runner (ubuntu-latest es el estándar).
- **Aislamiento total:** Los archivos de un job **no** existen en otro job a menos que se compartan como artefactos.

3. Dependencias entre Jobs (needs:)

Para estructurar un pipeline secuencial (*Stages*), se utiliza la directiva `needs:`.

```
jobs:
  test-backend:
    runs-on: ubuntu-latest
    ...
  test-frontend:
    runs-on: ubuntu-latest
    ...
  build-docker:
    needs: [test-backend, test-frontend] # Espera a que ambos terminen OK
    runs-on: ubuntu-latest
    ...
```

Regla de Oro: Si cualquier job predecesor falla, los jobs dependientes se **cancelan automáticamente**.

Un **Step** es una tarea ejecutable dentro del Job. Todos los steps de un job comparten el mismo entorno y disco.

```
steps:  
  - name: Descargar codigo fuente  
    uses: actions/checkout@v4  
  
  - name: Configurar Java 17 Temurin  
    uses: actions/setup-java@v4  
    with:  
      java-version: '17'  
      distribution: 'temurin'  
      cache: 'maven'  
  
  - name: Ejecutar pruebas unitarias  
    run: cd backend && mvn test
```

03

Comandos y Acciones

run: vs. uses: en la Práctica

run: vs. uses: ¿Cuándo usar cuál?

Comandos Shell (run:)

- Ejecuta comandos directos del CLI del SO (bash, sh).
- Ideal para compilar, probar o invocar herramientas locales:
 - `mvn clean test`
 - `npm run build`
 - `docker compose up -d`

Acciones Reutilizables (uses:)

- Módulos empaquetados del Marketplace de GitHub.
- Se configuran mediante parámetros en `with:`.
- Acciones esenciales:
 - `actions/checkout@v4`
 - `actions/setup-node@v4`
 - `docker/build-push-action@v5`

Como los runners son máquinas efímeras independientes, usamos artefactos para transferir archivos entre etapas:

```
# En Job 1: test-backend
- name: Subir artefacto JAR
  uses: actions/upload-artifact@v4
  with:
    name: backend-jar
    path: backend/target/*.jar
    retention-days: 1

# En Job posterior: deploy
- name: Descargar artefacto JAR
  uses: actions/download-artifact@v4
  with:
    name: backend-jar
    path: backend/target/
```

04

GitHub-Hosted vs. Self-Hosted

Entornos Gestionados vs. Servidores Propios

- **Agente Dedicado:** Software de GitHub (actions-runner) instalado en un servidor propio (ej. clúster lasLab grid102).
- **Acceso a Red Privada:** Puede conectarse directamente a bases de datos internas, LAN o VPN ZeroTier sin abrir puertos a Internet.
- **Hardware a la Medida:** Uso de servidores de alta capacidad (40 vCPUs, 62 GB RAM, GPUs para Deep Learning).
- **Cero Costo de Minutos:** Ejecuciones ilimitadas sin consumir minutos del plan de GitHub.

Casos de Uso Típicos

- Despliegues en servidores de intranet corporativa o universitaria.
- Tareas que requieren GPUs o hardware especializado.
- Caches masivas en disco local (evita descargas constantes).

Matriz Comparativa: GitHub-Hosted vs. Self-Hosted

Dimensión	GitHub-Hosted Runner	Self-Hosted Runner
Entorno & Ciclo	VM efímera (se destruye tras cada job). Disco 100 % limpio.	Host persistente. Workspace y archivos temporales permanecen.
Acceso a Red / LAN	Solo acceso público en Internet. Requiere VPN/túneles.	Acceso nativo a LAN, VPN (ZeroTier) y servicios internos (grid100, DBs).
Mantenimiento	GitHub actualiza el SO, parches y herramientas (Node, Java, Docker).	El administrador mantiene SO, Docker daemon, parches y herramientas.
Cómputo / Hardware	Estándar: 2 vCPU, 7 GB RAM. Hardware pre-determinado.	Personalizado (ej. 40 vCPUs, 64 GB RAM, GPUs NVIDIA en grid102).
Costos de Ejecución	Facturación por minutos consumidos según plan.	Gratuito en minutos de GitHub (costo solo de hardware propio).

Para enrutar un Job a un servidor propio, se modifica la directiva `runs-on`: utilizando etiquetas (*labels*):

```
# Opcion A: Runner estandar de GitHub
jobs:
  deploy-cloud:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: mvn clean test

# Opcion B: Runner Autohosteado en Servidor Propio (ej. grid102 IasLab)
jobs:
  deploy-grid100:
    runs-on: [ self-hosted, linux, x64, grid102 ]
    steps:
      - name: Limpiar workspace previo (por persistencia de disco)
        uses: actions/checkout@v4
        with:
          clean: true

      - name: Desplegar directamente en Docker local
        run: docker compose -f docker-compose.prod.yml up -d
```

Riesgos en Repositorios Públicos

- **¡Peligro con Forks y PRs!**

Un tercero podría enviar un PR con un script malicioso en el workflow que se ejecute en el servidor interno con acceso a la red de la empresa.

- **Persistencia de Procesos:**

Un proceso zombie o un contenedor huérfano puede interferir con jobs futuros.

Mejores Prácticas

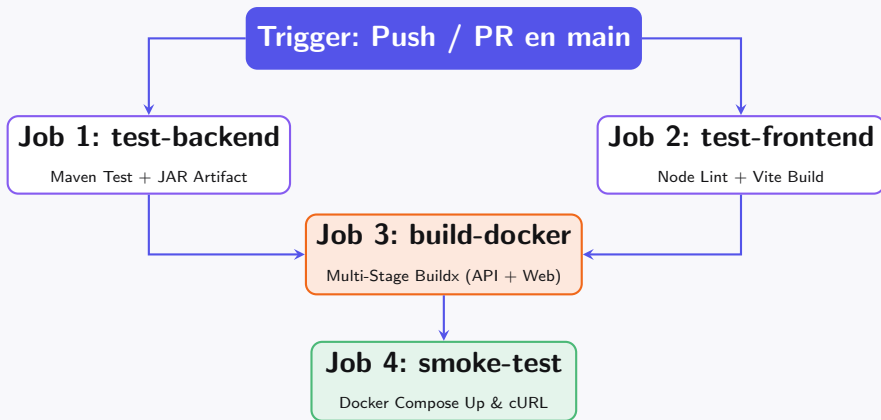
- Usar Self-Hosted **únicamente en repositorios privados** o de confianza interna.
- Requerir aprobación manual para ejecuciones provenientes de colaboradores externos.
- Utilizar **Runners Efímeros Contenerizados** (ej. *Actions Runner Controller* - ARC sobre Kubernetes).

05

Guía de Construcción del Pipeline

Estructura de los 4 Jobs del Taller

Estructura General del Pipeline a Construir



Paso 1: Configurar Encabezado y Triggers

Crear el archivo `.github/workflows/ci-cd.yml` y definir el nombre del flujo y los eventos:

```
name: CI/CD Pipeline - Product Application

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  # Aqui definiremos los 4 jobs de forma ordenada
```

Objetivo: Asegurar que el pipeline se dispare tanto al hacer `git push` directo como al abrir un *Pull Request*.

Responsable de validar la lógica del microservicio Spring Boot:

```
test-backend:
  name: Backend Unit Tests & Package
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Setup Java 17 (Eclipse Temurin)
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: 'maven'

    - name: Run Tests & Build JAR
      run: cd backend && mvn clean package

    - name: Upload Backend JAR
      uses: actions/upload-artifact@v4
      with:
        name: backend-jar
        path: backend/target/*.jar
```

Responsable de validar tipado TypeScript y empaquetar la SPA en React:

```
test-frontend:
  name: Frontend Lint & Build
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js 20
      uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: 'npm'
        cache-dependency-path: frontend/package-lock.json

    - name: Install & Run Lint
      run: |
        cd frontend
        npm install
        npm run lint

    - name: Build SPA Bundle
      run: cd frontend && npm run build
```

Construye las imágenes Docker utilizando *Docker Buildx*:

```
build-docker:
  name: Build Multi-Stage Docker Images
  needs: [test-backend, test-frontend]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build Backend Docker Image
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: false
        tags: product-api:latest

    - name: Build Frontend Docker Image
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: false
        tags: product-frontend:latest
```


Despliega el stack multicontenedor y valida su salud con pruebas de humo:

```
smoke-test:
  name: Compose Up & Smoke Testing
  needs: [build-docker]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Start Stack with Docker Compose
      run: docker compose up -d --build
    - name: Wait for Backend Healthcheck
      run: sleep 20
    - name: Execute Smoke Test (cURL API)
      run: |
        STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/api/products)
        echo "HTTP Status: $STATUS"
        if [ "$STATUS" -ne 200 ]; then exit 1; fi
    - name: Stop Containers
      if: always()
      run: docker compose down
```

100%

Despliegues Reproducibles y Automatizados

“Un pipeline de CI/CD bien estructurado convierte el despliegue de software en una tarea rutinaria, predecible y segura, eliminando la incertidumbre de los despliegues manuales.”

1. **Fallos Rápidos:** ¿Por qué es crucial ejecutar las pruebas unitarias antes de comenzar la construcción de imágenes Docker?
2. **Caché de Dependencias:** ¿Qué beneficio ofrece el uso de cache: 'maven' y cache: 'npm' en el tiempo de ejecución del runner?
3. **Self-Hosted vs. Cloud:** ¿En qué escenario de lasLab convendría usar un runner en grid102 en lugar de ubuntu-latest?
4. **Pruebas de Humo:** ¿Por qué validar un código HTTP 200 con curl nos da mayor certeza que solo verificar que el contenedor esté corriendo?

¡Éxitos en el desarrollo del Taller Evaluativo! (Tiempo: 2 Horas)